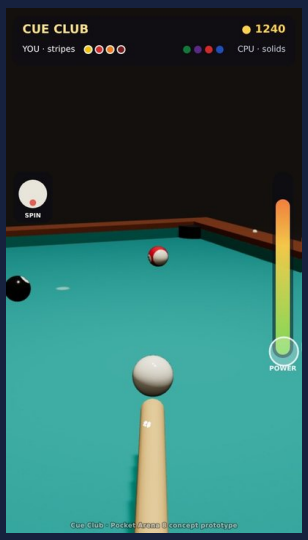
Pocket Arena 8

Build an 8-in-1 3D mini-game app with Claude Code

Pool · carrom · golf · penalties · snake · falling blocks · chess · city driving

iOS + Android · full 3D · solo vs CPU · shipped in three phases



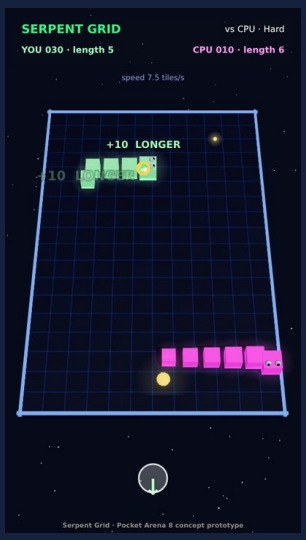
PHASE 1 **Cue Club**



PHASE 1 **Stack Drop**



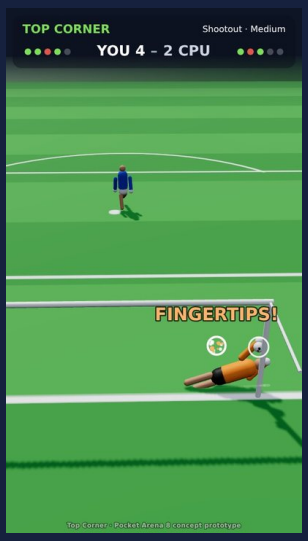
PHASE 1 **Chess 3D**



PHASE 1 **Serpent Grid**



PHASE 2 **Carrom Corner**



PHASE 2 **Top Corner**



PHASE 3 **Fairway Flick**



PHASE 3 **Park & Pass**

Frames come from the 3D concept prototypes in this pack (prototypes/*.html, gameplay/*.mp4). Edition: November 2026.

How to use this pack

This guide plus the zip next to it give Claude Code everything it needs to build **Pocket Arena 8**: one app, eight original 3D mini-games, all played against the computer. The zip holds a ready Claude Code project kit (CLAUDE.md, path-scoped rules for each game, skills, subagents, hooks), the design documents, and 10-second 3D gameplay clips with keyframes Claude can look at.

What is in the zip

```
pocket-arena-8-pack/
  PocketArena8_Build_Guide.pdf          <- this document
  README.md   FEED_TO_CLAUDE.md          <- quick start + copy-paste commands
  gameplay/
    00_all_games_reel.mp4                <- hub + all 8 games (87 s)
    00_pocket_arena_hub.mp4, 01_cue_club.mp4 ... 08_park_and_pass.mp4   (540x960, 30 fps)
    gif/*.gif   keyframes/*.png          <- previews; 3 stills per clip (Claude can view PNGs)
  prototypes/                            <- three.js concept mocks the clips were recorded from
    *.html (open in a browser to watch them loop)   src/*.js   lib/harness.js
  claude-kit/                            <- copy this to start the real Godot project
    CLAUDE.md   .mcp.json.example   ASSETS.md
    .claude/settings.json  .claude/hooks/test_gate.sh  .claude/hooks/format_lint.sh
    .claude/rules/{hub,cue-sim,blocks,chess,snake,golf,penalty,drive}.md
    .claude/skills/{new-minigame,physics-tune,ai-opponent,perf-check,shot,release-android}/SKILL.md
    .claude/agents/{physics-reviewer,test-writer,ip-checker,asset-wrangler}.md
    docs/GDD.md  docs/games/<id>.md  docs/STARTER_PROMPTS.md  docs/KICKOFF_PROMPT.md  docs/PROGRESS.md
    docs/refs/<id>/{keyframe_1..3.png, clip.mp4, prototype.html}
  scripts/start_project.sh, start_project.ps1
```

Feed it to Claude Code

Claude Code reads PDFs (in page ranges for long ones; install **poppler**) and PNG images with its Read tool. It cannot watch .mp4 files, which is why every clip also has keyframe PNGs and the prototype source. Copy commands from README.md rather than this PDF, since long lines wrap here.

```
unzip pocket-arena-8-pack.zip && cd pocket-arena-8-pack
./scripts/start_project.sh ~/games/pocket-arena-8
#   copies claude-kit + this PDF into a new folder, runs git init, and opens Claude Code in plan mode
#   with docs/KICKOFF_PROMPT.md (read CLAUDE.md, GDD, refs -> interview you -> plan week 1; no code yet)

# or explore first, from the unzipped folder:
claude "Read README.md and PocketArena8_Build_Guide.pdf in page ranges, look at
    gameplay/keyframes/*.png, then summarise the plan and ask me what to start with."
```

Then work through **docs/STARTER_PROMPTS.md** in order: one prompt per session, plan mode for anything touching more than three files. Start each later session with "*Read CLAUDE.md and docs/PROGRESS.md, then continue with the next starter prompt.*"

Contents

Section	Page
How to use this pack	2
The short version	4
Legal naming and Claude Code's game rule	5
Engine choice: Godot 4.7 + Jolt	6
Hub architecture	7
The shared physics core	8
Phase 1 · Cue Club — 8-ball pool vs CPU	9
Phase 1 · Stack Drop — Original falling-block puzzle vs CPU	11
Phase 1 · Chess 3D — Chess vs CPU	13
Phase 1 · Serpent Grid — Snake vs CPU	15
Phase 2 · Carrom Corner — Carrom vs CPU	17
Phase 2 · Top Corner — Penalty shootout vs CPU	19
Phase 3 · Fairway Flick — Golf vs CPU	21
Phase 3 · Park & Pass — Original city driving & parking missions	23
Claude Code setup for this project	25
3D asset pipeline	27
Phased roadmap, monetization and publishing	28
Risks, scope cuts and caveats	29
Sources	30

The short version

Build **one Godot 4.7 app (GDScript, Jolt physics, Mobile renderer)** with a small hub and eight game modules loaded one at a time. Spend your one serious physics investment on a **deterministic custom ball/disc solver** shared by pool and carrom (and their AI); use Jolt rigid bodies only for golf landings, the football after contact and the car; snake, the block puzzle and chess need no physics at all.

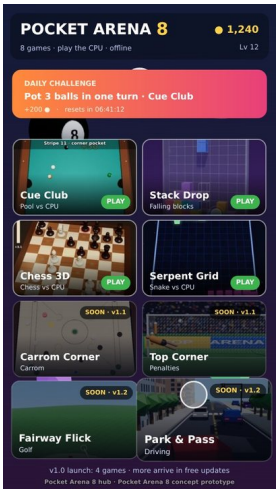
Decision	Recommendation
Your choices	Solo + vs CPU (no servers) · full 3D · iOS + Android · ship in phases
Engine	Godot 4.7.x + GDScript + Jolt (default 3D physics since 4.6). Text scenes, headless tests, MCP servers → best fit for Claude Code. Runner-up: Unity 6 (official Claude Code plugin, stronger ads/IAP SDKs).
Phase 1 (~12 weeks)	Hub + Cue Club (pool) + Stack Drop (blocks) + Chess 3D + Serpent Grid (snake)
Phase 2 (~6 weeks)	Carrom Corner (reuses the pool solver) + Top Corner (penalties) + daily cross-game challenges
Phase 3 (~10 weeks)	Fairway Flick (golf) + Park & Pass (driving) — the highest physics and content risk
Chess AI	Your own alpha-beta engine. Not Stockfish: it is GPLv3 and conflicts with a closed-source store app.
Legal	Original names, art, UI and music. Never "Tetris"/"-tris", "8 Ball Pool", or a "Dr." prefix. Prompt Claude by mechanics.
Store	New Google Play personal accounts: closed test with 12+ testers opted in for 14 continuous days. Start it during development.

Why this Phase 1 lineup

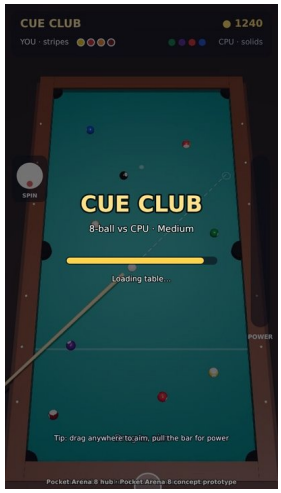
- **Pool** is the demand anchor, and its solver is reused by carrom, so the physics risk is paid once.
- **Chess, snake and blocks** are low physics risk and high replay value; they let the hub, save system, economy and ads stabilise quickly.
- **Golf and driving** carry the most content and physics risk, so they come last.
- Successful mini-game collections share habits worth copying: one strong anchor game, a single soft currency, cosmetics that carry across games, daily challenges rotating across games, 60–180 s sessions, and at most two taps from launch to gameplay.

Gameplay clips (10 s each) and the hub

The pack includes a short hub clip (launcher with four Phase 1 games playable and four marked for later updates) plus one clip per game. Each game chapter shows three stills and describes the clip.







00_pocket_arena_hub.mp4
Cards slide in; v1.0 shows Cue Club, Stack Drop, Chess 3D and Serpent Grid as playable; Carrom Corner and Top Corner are tagged "SOON · v1.1", Fairway Flick and Park & Pass "SOON · v1.2". A tap on Cue Club zooms into its loading screen.

Legal naming and Claude Code's game rule

Practical guidance, not legal advice. Mechanics (falling blocks, 8-ball rules, parking missions) are generally not protected; names, logos, distinctive visual expression and UI layouts can be. Run a trademark search (USPTO, EUIPO and your home market) on the hub name and each game name before listing.

Inspiration	Risk	Avoid	Use instead
Tetris	High: trademark + trade dress (brightly coloured blocks, tall rectangular playfield); Tetris Holding v. Xio (D.N.J. 2012) found infringement on look-and-feel	The name, '-tris', 'tetromino' in marketing, a 10×20 well with classic flat colours, the folk melody	Stack Drop (Block Cascade, Crystal Tower)
8 Ball Pool (Miniclip)	Medium: '8 BALL POOL' is a registered EU/UK mark; the '8 POOL' flame logo is registered in the US	'8 Ball Pool', '8 Pool', the flame logo, their UI layouts and level/ring progression look	Cue Club (Corner Pocket 3D, Break & Bank)
Dr. Driving (SUD Inc.)	Medium: the 'Dr.' series is SUD's brand identity across many titles	The 'Dr.' prefix, their dashboard-style HUD layout, mission names	Park & Pass (City Driver Academy, Lane Master)
Carrom, chess, snake, golf, penalties	Low: generic names	Copying a specific app's art or UI	Carrom Corner, Chess 3D, Serpent Grid, Fairway Flick, Top Corner

Practical rules

1. Make every visual, sound and UI layout original (Stack Drop uses a 9×18 glass tower with gem-shaded "shards", not a 10×20 well of flat bright squares).
2. Never mention competitors in the store listing or ASO keywords.
3. Keep design docs that show your independent choices (the kit's docs/games/*.md are a start).
4. Run the **ip-checker** subagent before every release.

How Claude Code handles "make me X"

- In late October 2026 Anthropic added a server-side Claude Code rule: build original games, games in a genre, and games from the user's own designs and assets, but not a recreation of a specific existing game, even renamed. Anthropic said the reason is that using Claude to infringe IP is against its usage policy.
- Version 2.1.317 (early November 2026) narrowed it: you can name a game to describe what you want, and Claude will build an original game in that genre.
- So write specs and prompts in mechanics language: "*an original falling-block puzzle with SRS-style rotation and a 7-bag randomizer, rendered as a 3D crystal tower*", not "*make Tetris*". Your own designs and assets are explicitly in scope, and the kit's CLAUDE.md and rules files are already written this way.

Engine choice: Godot 4.7 + GDScript + Jolt

Criterion	Godot 4.7 + Jolt	Unity 6	Web 3D + Capacitor	Unreal / Defold / Flame
AI / CLI friendliness	Excellent: text .tscn/.tres/.gd, --headless, CLI export, several MCP servers	Very good: official Claude Code plugin (29 skills), Unity MCP; verbose GUID-heavy YAML scenes	Excellent (plain TS)	Unreal: binary assets; Defold/Flame: thin 3D
Billiards / carrom	Custom solver recommended (any engine)	Same advice	Rapier/Havok WASM	—
Vehicles	VehicleBody3D raycast vehicle or custom	WheelCollider (best out of the box)	Plugins	Unreal Chaos (heavy)
Mobile perf / size	Light runtime; strip with custom templates	Larger, mature	Weakest on low-end Android	Unreal heavy
Ads / IAP	Community plugins (thinner)	First-party, strongest	Capacitor plugins	—
Cost	MIT, free	Free tier with thresholds	Free	Unreal 5% royalty above threshold

Why Godot wins for this project

- **Agent ergonomics decide it.** Claude can create, review and diff scenes without an editor, run games and tests headless, and use MCP servers for live run/screenshot loops.
- **Physics is now good enough.** Jolt arrived in 4.4 as experimental; with 4.6 (January 2026) the Godot team declared it production-ready and made it the default for new 3D projects. 4.7 shipped in June 2026 (4.7.2 is the current maintenance release).
- **4.7 helps directly:** a built-in VirtualJoystick node (Fixed / Dynamic / Following) for driving, and 3D physics interpolation that decouples physics ticks from frame rate on mobile.
- **Use GDScript, not C#**: Godot's docs still describe C# Android export as experimental.
- **Choose Unity instead** if you already know C#, want first-party ad mediation/IAP with the least friction, or want WheelCollider for driving.

Project settings to fix on day one

```
physics/3d/physics_engine = "Jolt Physics"     # default in new 4.6+ projects
physics/common/physics_ticks_per_second = 60
physics/common/physics_interpolation = true
rendering/renderer/rendering_method = "mobile"  # Compatibility renderer as the low-end fallback
rendering/textures/vram_compression/import_etc2_astc = true
# Export: Android AAB (Gradle build) for Google Play; iOS exports an Xcode project (needs a Mac)
```

Hub architecture

```
res://
  hub/                  # launcher, menus, shop, settings UI
  core/                 # autoload singletons (shared services)
    SaveService.gd  SettingsService.gd  AudioService.gd  InputKit.gd  CameraKit.gd  Economy.gd
    Achievements.gd  Ads.gd  IAP.gd  Analytics.gd  GameRegistry.gd  SceneRouter.gd  Rng.gd
  shared/
    cue_sim/            # deterministic disc/ball solver (pool + carrom + their AI)
    aim_ui/  power_meter/  ai_common/
  games/
    pool/ carrom/ golf/ penalty/ snake/ blocks/ chess/ drive/
      <game>.tscn  manifest.tres  logic/  scenes/  assets/  tests/
  tests/                # hub-level tests
```

Service	Responsibility
GameRegistry	Reads each games//manifest.tres (id, display name, scene, min save version, enabled flag, unlock cost). The hub lists registered + enabled games, so adding a game = a folder + manifest, no hub code.
SceneRouter	Loads one game with ResourceLoader.load_threaded_request() behind a loading screen and frees the previous one, so only one game's assets are in memory.
SaveService	Single versioned user://save.json; each game owns its namespace + schema + migrate(); unknown keys are preserved; temp file → atomic rename → one rolling backup. Old saves simply lack new games' keys, so updates are safe.
Economy	Coins, cosmetic ownership, equipped skins (cue, striker, ball trail, car paint, chess set, block material).
InputKit	Drag-to-aim, pull-back power, swipe, tap, pinch-zoom, two-finger orbit, and Godot 4.7's VirtualJoystick.
CameraKit	Three rigs: top-down table (pool/carrom/chess), orbit-follow (golf/driving), fixed broadcast (penalty/snake/blocks); smoothing, spring arms, shake.
Ads / IAP / Analytics	Thin interfaces with no-op stubs in debug and tests, so Claude tests game logic without SDKs.

Save file shape

```
{ "schema": 3, "profile": {...}, "economy": {"coins": 1240, "owned": [...], "equipped": {...}},
  "games": { "pool": {"schema": 1, "wins": 12, ...}, "chess": {"schema": 2, ...} } }
```

Size and performance budgets

- Ship new games as normal store updates (no servers). Keep each game's assets under ~25 MB and the whole app under ~150 MB; Godot AAB exports apply install-time Play Asset Delivery automatically, so on-demand delivery is only needed if you outgrow the base module.
- Feature-flag unfinished games (enabled=false) so half-built work can live on the main branch.
- Low-end Android: < 100 draw calls, < 150k visible triangles, one directional light, baked or blob shadows, no real-time GI, < 150 MB texture memory, 30 fps floor (60 fps target on mid-range), Compatibility renderer on GLES3-only devices.
- General rules for every game: fixed-timestep logic, seeded RNG (Rng.gd) so AI and tests are reproducible, game logic in plain GDScript classes separate from nodes, never multiply by render delta inside a simulation step.

The shared physics core

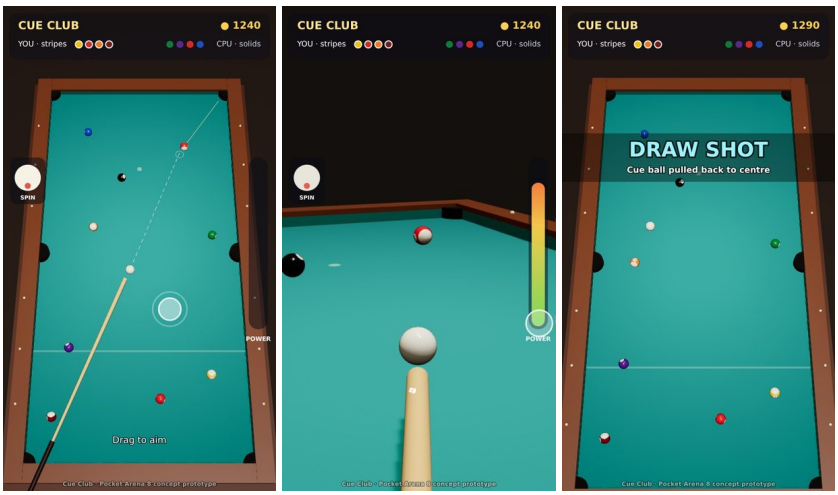
The one serious physics investment is **shared/cue_sim**: a deterministic 2.5D solver for balls and discs. It moves bodies in the table plane with 3D spin, in fixed 1/240 s substeps, with analytic sliding and rolling phases and swept-circle (time-of-impact) collisions processed in time order. Pockets are geometric capture tests. Rendering simply draws its state on 3D meshes.

- **AI becomes cheap:** the CPU can simulate thousands of candidate shots headlessly using the exact same code as the player's shot.
- **Tests become exact:** golden-master breaks, stop shots and energy checks are deterministic to 1e-6.
- **No engine jitter:** no rigid-body stacking or sleeping issues for resting balls and coins.
- **Carrom is nearly free:** "disc mode" drops the rolling phase and swaps constants.

The pool and carrom clips in this pack were recorded from a small JavaScript version of exactly this idea (prototypes/lib/harness.js, class DiscSim): constant-deceleration friction, restitution for ball–ball and rails, fixed substeps and pocket capture. It is a visual mock, not the production solver, but it shows why scripted shots like "queen, then cover" are reproducible every run.

Phase 1 Cue Club

8-ball pool vs CPU — the anchor game



At a glance	
Phase	Phase 1
Genre	8-ball pool vs CPU
Build	4–5 weeks
Risk	Medium–High (custom physics)
Rules file	.claude/rules/cue-sim.md

In the 10-second clip (gameplay/01_cue_club.mp4): Drag to aim (dotted line + ghost ball), pull the power bar, release. The camera swoops low behind the cue; stripe 11 is cut into the corner pocket (+50), the cue ball draws back to the centre with backspin — 'DRAW SHOT', 'YOUR TURN AGAIN'.

Physics approach

Custom deterministic solver (shared/cue_sim): balls move in the table plane with 3D spin; fixed 1/240 s substeps; analytic sliding → rolling phases; swept-circle (time-of-impact) collisions so nothing tunnels. Jolt is not used for ball motion; pockets are geometric capture tests.

Tuning starting points

Parameter	Value
Sliding cloth friction μs	≈ 0.2 (measured 0.178–0.245)
Rolling friction μr	≈ 0.01–0.016
Rolling deceleration	5/7 · μr · g
Ball–ball restitution	≈ 0.93
Ball–ball friction	≈ 0.05–0.06 (gives throw and cut-induced spin)
Cushion restitution	start 0.85, tune by feel (published values conflict: 0.82 measured vs 0.98 in one model)
Cue tip offset	clamp spin input to 0.5 R (avoids miscues)
Balls / table	57.15 mm, ~170 g; 9-ft playfield 2.54 × 1.27 m

Rules and modes

8-ball state machine: open table → groups assigned on first legal pot → fouls (scratch, wrong ball first, no rail) → ball-in-hand → 8-ball win/loss (early 8 = loss).

CPU opponent

Enumerate (object ball, pocket) pairs → ghost-ball aim → sample N power/spin values → simulate each to rest with the headless solver → score: pot success, cue-ball position value (how many next shots), scratch penalty, safety value. Easy N=8, ±2.5° aim noise, no position play; Medium N=32, ±1.0°; Hard N=128, ±0.3° plus two-ply lookahead. Budget ~150 ms spread across frames behind a 'thinking' camera pan.

Controls and camera

Controls: Drag anywhere to rotate aim (fine-aim slider on the right edge); pull the vertical power bar and release; spin dot on a cue-ball inset for english.

Camera: Top-down 3/4 by default → lerps low behind the cue during the stroke → follow cam on the fastest ball.

Tests Claude should write first

- Golden-master break: fixed seed + cue velocity → identical final positions to 1e-6.
- Energy never increases during a shot.
- Straight stop shot leaves the cue ball within 1 cm.
- 10,000 random shots → no ball overlaps.

Pitfalls: Frame-rate-dependent friction; not sorting multiple collisions inside one substep by time of impact; rule-state bugs (open table, fouls, early 8). **If behind:** Ship without spin (centre-ball only) if behind.

Path-scoped rule file: .claude/rules/cue-sim.md

```
---
paths: ['shared/cue_sim/**', 'games/pool/**', 'games/carrom/**']
---
- Solver is deterministic: fixed 1/240 s substeps, swept-circle time-of-impact, process collisions in
    time order.
- Pool defaults: mu_slide 0.2, mu_roll 0.012, e_ball 0.93, mu_ball 0.055, e_cushion 0.85. Carrom: disc
    mode, no rolling
  phase, mu_k 0.1, e_disc 0.8, e_rail 0.65, striker mass 2.7x coin.
- Any change must keep tests/golden_break_test.gd passing or update the golden file with a written
    justification.
- AI shot search calls the same solver headlessly; budget 150 ms spread across frames.
```

Starter prompts (one per session, plan mode first)

1. Read CLAUDE.md and .claude/rules/cue-sim.md. In plan mode, design shared/cue_sim: a deterministic 2.5D ball solver with a sliding→rolling transition, swept-circle time-of-impact collisions, cushions and pocket capture. List the tests before any code.
2. Implement the solver and its tests: golden-master break, stop shot, energy never increases, 10k random shots with no overlap. Run them headless.
3. /new-minigame pool — then build a low-poly 3D table scene (original felt colour and rail style) that renders the solver state.
4. Implement the 8-ball rules state machine with tests: open table, groups, fouls, ball-in-hand, 8-ball win/loss.
5. Build the touch aim / power / spin UI on InputKit, then /shot pool and review the screenshots.
6. /ai-opponent pool easy|medium|hard — report self-play win rates against the targets.
7. /perf-check and fix any budget violations.

Phase 1 Stack Drop

Original falling-block puzzle vs CPU — original design, legal care



At a glance	
Phase	Phase 1
Genre	Original falling-block puzzle vs CPU
Build	1.5–2 weeks
Risk	Low physics · HIGH legal care
Rules file	.claude/rules/blocks.md

In the 10-second clip (gameplay/06_stack_drop.mp4): A purple shard is dragged against the glass wall and rotated (WALL KICK), hard-dropped; a long sapphire shard drops into the gap and four rows flash and shatter into crystal fragments — 'QUAD CLEAR', +4 garbage rows hit the CPU's tower.

Physics approach

No physics. A pure functional core reduce(state, action) with a seeded RNG, so any game replays bit-exactly.

Tuning starting points

Parameter	Value
Rotation	SRS-style wall-kick tables (5 tests per rotation; separate table for the long piece)
Randomizer	7-bag: each of the 7 shards once per shuffled bag
Lock delay	0.5 s with move-reset, capped at 15 resets
Assist	ghost piece, hold, 3-piece preview
Scoring	100 / 300 / 500 / 800 × level; level up every 10 lines
Touch timing	DAS 170 ms, ARR 50 ms

Rules and modes

Original design — mechanics aren't protected, expression is: a 9-wide × 18-tall glass 'crystal tower', physically shaded gem materials instead of flat bright colours, pieces named shards A–G, your own music. Optional twist: the tower slowly rotates around a cylinder.

CPU opponent

Side-by-side race. Classic placement heuristic (aggregate height, holes, bumpiness, lines cleared) evaluated over all rotations/columns; clears send garbage rows to the opponent.

Controls and camera

Controls: Tap = rotate, horizontal drag = move (DAS/ARR), swipe down = hard drop, two-finger tap = hold.

Camera: Fixed, slightly tilted view of the tower with a slow orbit; CPU's tower as a mini-view.

Tests Claude should write first

- 7-bag never repeats a shard within a bag.
- Kick tables: every rotation test case from the spec.
- Replays: same seed + inputs → identical board hash.

Pitfalls: Never use the 10×20 well with flat bright squares, the trademarked name or '-tris', or the classic folk melody associated with it. **If behind:** Drop vs-CPU; ship score-attack only.

Path-scoped rule file: .claude/rules/blocks.md

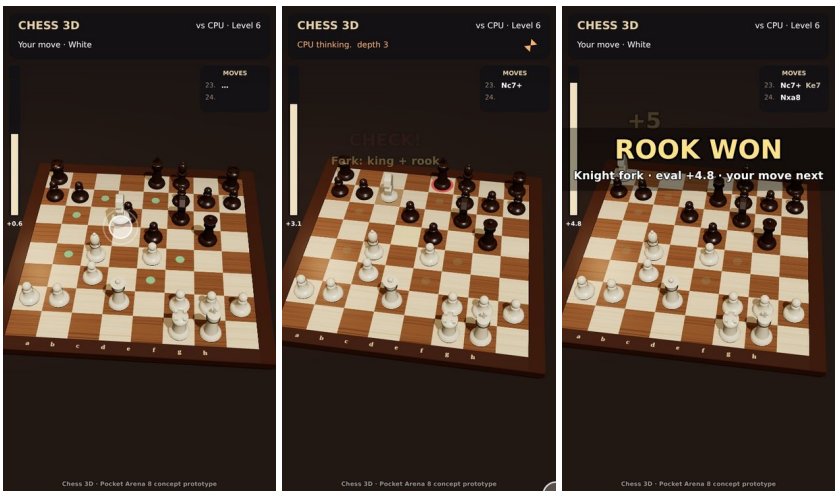
```
---
paths: ['games/blocks/**']
---
- Original falling-block puzzle "Stack Drop": 9x18 crystal tower, gem-shaded pieces, our own audio.
- Never use the words Tetris/tetromino in code, comments, UI or assets. Pieces are "shards" A-G.
- Core is reduce(state, action) with a seeded 7-bag; replays must be bit-exact.
- SRS-style kick tables live in data, with one test per kick case.
```

Starter prompts (one per session, plan mode first)

1. Read CLAUDE.md and .claude/rules/blocks.md. In plan mode, design the pure reducer: state, actions, SRS-style kicks, seeded 7-bag, lock delay. Tests first.
2. Implement the reducer + tests for kicks, the bag and bit-exact replays.
3. /new-minigame blocks — render the reducer as a 3D crystal tower (original look: 9×18, gem materials, shard names A–G).
4. Add touch controls with DAS/ARR, hard drop, hold; capture /shot blocks and check readability on a small phone.
5. Add the CPU rival (heuristic placer) with garbage rows; /ai-opponent blocks medium.

Phase 1 Chess 3D

Chess vs CPU — your own engine



At a glance	
Phase	Phase 1
Genre	Chess vs CPU
Build	2–3 weeks
Risk	Low physics · licence care
Rules file	.claude/rules/chess.md

In the 10-second clip (gameplay/07_chess_3d.mp4): Tap the knight (legal-move dots appear), drag it to c7 — CHECK, a fork on king and rook. The CPU 'thinks' (depth 3), steps the king away, and the knight leaps to a8; the rook flies off the board — 'ROOK WON', eval +4.8.

Physics approach

No physics. Bitboard or 0x88 move generator in GDScript (castling, en passant, promotion, threefold repetition, 50-move rule).

Tuning starting points

Parameter	Value
Validation	perft node counts — perft(start, 4) = 197,281
Engine	negamax alpha-beta, iterative deepening, quiescence, MVV-LVA ordering, transposition table, piece-square tables
Expected depth	3–4 in GDScript within a time budget; move search to a C++ GDExtension later if needed
Easy	depth 1, random among top-4 within 150 cp
Medium	depth 3
Hard	depth 4–5 + quiescence

Rules and modes

Full FIDE rules. Do NOT embed Stockfish: it is GPLv3 (you must ship the licence and full source), and the FSF has said Apple's App Store terms conflict with the GPL. It only works if the whole app is open-sourced under the GPL. Write your own engine; check the licence of any engine you consider.

CPU opponent

Your own negamax alpha-beta engine; difficulty via depth/time limits plus random choice among the top-k moves within an evaluation margin.

Controls and camera

Controls: Tap a piece then a destination, or drag; legal-move dots; last-move highlight.

Camera: 3D board at ~50° with swipe-orbit, plus a top-down toggle.

Tests Claude should write first

- perft from the start position and standard test positions.
- Mate-in-1 / mate-in-2 puzzles solved at Medium.
- Engine never returns an illegal move (fuzz 10k random positions).

Pitfalls: Slow GDScript search on low-end phones — run the search on a thread and cap by time, not depth. **If behind:** Ship without the 3D orbit (fixed camera) and with two difficulty levels.

Path-scoped rule file: .claude/rules/chess.md

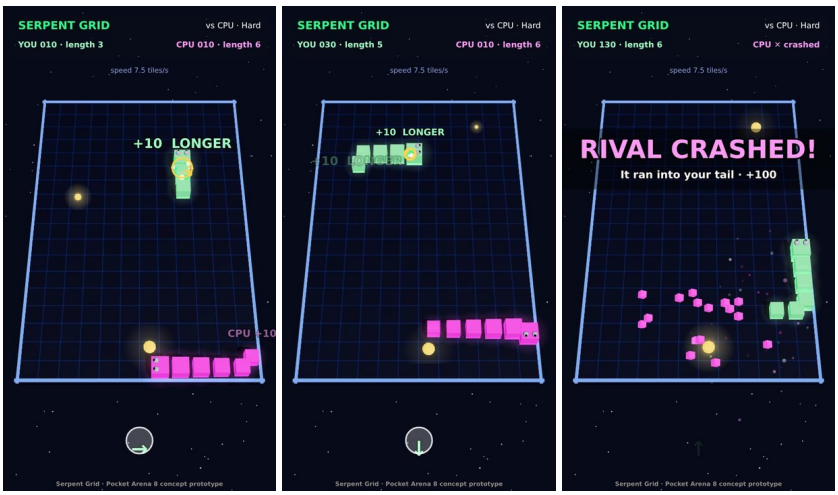
```
---
paths: ['games/chess/**']
---
- Do NOT add Stockfish or any GPL code. Engine is our own negamax alpha-beta in
    games/chess/logic/engine.gd.
- perft(start, 4) must equal 197281; keep tests/perft_test.gd green.
- Search runs on a WorkerThreadPool task with a time budget; UI never blocks.
```

Starter prompts (one per session, plan mode first)

1. Read CLAUDE.md and .claude/rules/chess.md. Plan the move generator and the perft test harness first.
2. Implement the move generator until perft(start, 1..4) matches; add standard test positions.
3. Implement negamax alpha-beta with iterative deepening, TT, quiescence and MVV-LVA; add mate-in-N tests.
4. Add difficulty knobs (depth/time/top-k margin) and a threaded search with a time budget.
5. /new-minigame chess — 3D board, lathe-turned pieces, tap/drag input, legal-move dots; /shot chess.

Phase 1 Serpent Grid

Snake vs CPU — the quickest win



At a glance	
Phase	Phase 1
Genre	Snake vs CPU
Build	~1 week
Risk	Low
Rules file	.claude/rules/snake.md

In the 10-second clip (gameplay/05_serpent_grid.mp4): A glowing green snake turns with swipes across a floating neon grid, eats two orbs and grows; the magenta CPU snake loops below, misjudges a turn and crashes into your tail — 'RIVAL CRASHED!'.

Physics approach

No physics. Discrete grid logic (e.g., 12×18 or 20×20) rendered as 3D blocks on a floating board, smoothly interpolated between ticks.

Tuning starting points

Parameter	Value
Tick rate	starts ~7 Hz, rises with length
Input	swipe to turn, 2-deep input buffer so quick double swipes register
Camera	tilted ~55°
Optional mode	board wrapped around a cube

Rules and modes

Eat orbs to grow; hitting a wall, yourself or the rival ends a run. Vs-CPU: first to crash loses.

CPU opponent

Rival snake: A* to the nearest orb, rejected if a flood-fill shows it would trap itself (safety check).

Controls and camera

Controls: Swipe anywhere to turn.

Camera: Fixed tilted view of the whole board.

Tests Claude should write first

- Pure-logic tests: collision, growth, input buffer, wrap mode.
- AI never chooses a move that flood-fill marks as a trap.

Pitfalls: Interpolating rendering at a different rate from logic ticks; dropped swipes without the buffer.
If behind: Score-attack only (no rival).

Path-scoped rule file: .claude/rules/snake.md

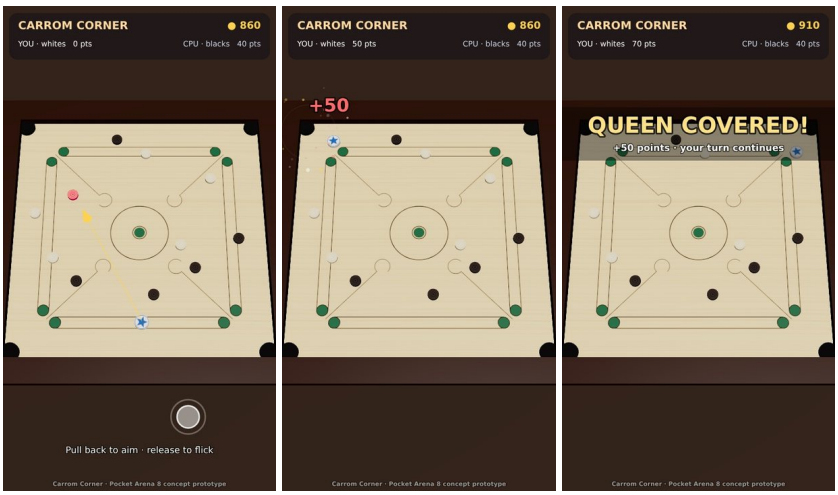
```
---
paths: ['games/snake/**']
---
- Pure grid logic in games/snake/logic/; renderer interpolates between ticks and never owns state.
- 2-deep swipe buffer; tick rate from data (start 7 Hz).
- CPU rival: A* to orb + flood-fill safety check; never choose a move flood-fill marks as a trap
    (tested).
```

Starter prompts (one per session, plan mode first)

1. Read CLAUDE.md and .claude/rules/snake.md. Implement pure grid logic with tests (collision, growth, 2-deep input buffer).
2. /new-minigame snake — 3D renderer with smooth tick interpolation and neon materials.
3. Add swipe input via InputKit and the rival snake (A* + flood-fill safety) with tests.

Phase 2 Carrom Corner

Carrom vs CPU — reuses the pool solver



At a glance	
Phase	Phase 2
Genre	Carrom vs CPU
Build	2.5–3 weeks (after pool)
Risk	Medium (reuses pool core)
Rules file	.claude/rules/cue-sim.md

In the 10-second clip (gameplay/02_carrom_corner.mp4): The striker slides along the baseline, a slingshot pull aims it, and it cracks into the queen, which drops into the corner — 'QUEEN POCKETED'. Next shot a white coin covers her — 'QUEEN COVERED! +50'.

Physics approach

Reuses shared/cue_sim in 'disc mode': discs slide and don't roll (kinetic friction only), small yaw spin for visuals.

Tuning starting points

Parameter	Value
Disc–board friction μk	≈ 0.08–0.12 (powdered board)
Disc–disc restitution	≈ 0.75–0.85
Rail restitution	≈ 0.6–0.7
Masses	striker ~15 g, coins ~5.5 g (≈2.7×)
Sizes	74 cm playing surface, 4.45 cm pockets, 41.3 mm striker, 31.8 mm coins

Rules and modes

Turn order, queen + 'cover' rule, fouls (striker pocketed, coin returned), baseline striker-placement validation.

CPU opponent

Same simulation search as pool; candidate shots = direct, single rail rebound and cuts; difficulty by noise and sample count.

Controls and camera

Controls: Slide the striker along the baseline (horizontal drag), drag back from it to aim and set power (slingshot), release to flick.

Camera: Fixed top-down, rotated so your baseline is at the bottom, ~15° tilt for depth.

Tests Claude should write first

- Disc-mode golden master (no rolling phase).
- Striker placement always inside the baseline.
- Queen/cover rule cases.

Pitfalls: Disc stacking jitter if you use a rigid-body engine instead of the shared solver. **If behind:** No 'thumb' cut shots for the AI in v1.

Path-scoped rule file: .claude/rules/cue-sim.md

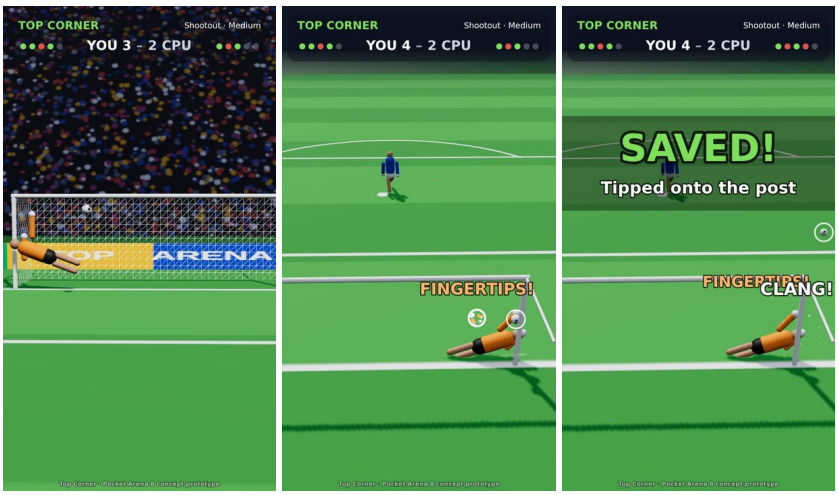
```
---
paths: ['shared/cue_sim/**', 'games/pool/**', 'games/carrom/**']
---
- Solver is deterministic: fixed 1/240 s substeps, swept-circle time-of-impact, process collisions in
    time order.
- Pool defaults: mu_slide 0.2, mu_roll 0.012, e_ball 0.93, mu_ball 0.055, e_cushion 0.85. Carrom: disc
    mode, no rolling
  phase, mu_k 0.1, e_disc 0.8, e_rail 0.65, striker mass 2.7x coin.
- Any change must keep tests/golden_break_test.gd passing or update the golden file with a written
    justification.
- AI shot search calls the same solver headlessly; budget 150 ms spread across frames.
```

Starter prompts (one per session, plan mode first)

1. Read .claude/rules/cue-sim.md. Extend shared/cue_sim with disc mode (no rolling phase) + tests.
2. /new-minigame carrom — board scene, coins, striker; rules engine (turns, queen/cover, fouls) with tests.
3. Baseline striker placement + slingshot UI; then /ai-opponent carrom medium reusing the pool search.

Phase 2 Top Corner

Penalty shootout vs CPU — swipe, curl, dive



At a glance	
Phase	Phase 2
Genre	Penalty shootout vs CPU
Build	2–3 weeks
Risk	Medium
Rules file	.claude/rules/penalty.md

In the 10-second clip (gameplay/04_top_corner.mp4): A curved swipe bends the ball around the keeper's full-stretch dive into the top corner; slow-motion net ripple — 'GOAL!'. Then you keep: swipe to dive, fingertips push the CPU's shot onto the post — 'SAVED!'.

Physics approach

Custom flight integrator (as for golf) for drag + Magnus; after any contact (post, keeper, net) the ball becomes a Jolt RigidBody3D. Net = a few low-restitution StaticBody3D panels, or a vertex shader for visual flex.

Tuning starting points

Parameter	Value
Ball	0.22 m, ~0.43 kg
Drag crisis	blend Cd 0.45 → 0.2 across ~15–20 m/s (critical Re 2.2–3.0×10^5)
Magnus side force	coefficient ~0.2–0.3; exaggerate ~1.5× for fun
Keeper reaction	Easy 320 ms · Medium 220 ms · Hard 150 ms
Keeper reach	2.6–3.2 m dive; error ±0.5 / 0.3 / 0.15 m
Hard mode	pre-commits ('guesses early') 30% of the time

Rules and modes

You shoot five, then keep five (swipe to dive), then sudden death. Free-kick-with-wall challenge as a later add-on.

CPU opponent

Keeper state machine Idle → Read (reaction delay) → Commit dive → Recover; predicts the crossing point each frame with the same integrator. CPU striker picks corners with a weighted random + your recent dive history.

Controls and camera

Controls: Swipe from the ball: direction = aim, length/time = power, curvature (quadratic fit of the touch points) = curl. Keeping: swipe to dive.

Camera: Broadcast angle behind the shooter; elevated behind-the-goal view when keeping.

Tests Claude should write first

- Integrator: a straight swipe produces no lateral drift.
- Keeper difficulty table matches spec (reaction/reach).
- Swipe-curve fit on recorded touch fixtures.

Pitfalls: Frame-rate-dependent integration; keeper that is perfect (always add reaction delay and reach error). **If behind:** Shooting only (no keeping) in the first update.

Path-scoped rule file: .claude/rules/penalty.md

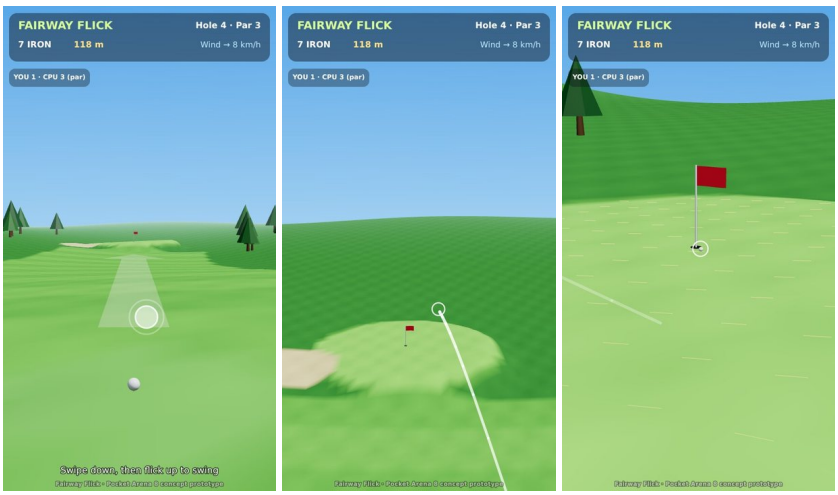
```
---
paths: ['games/penalty/**']
---
- Ball 0.22 m / 0.43 kg. Cd blends 0.45 -> 0.2 across 15-20 m/s. Magnus side coefficient 0.25 (x1.5 for
    fun).
- Keeper table (reaction ms / reach error m): Easy 320/0.5, Medium 220/0.3, Hard 150/0.15 + 30% early
    guess.
- After any contact the ball becomes a RigidBody3D; net panels are low-restitution StaticBody3D.
```

Starter prompts (one per session, plan mode first)

1. Read .claude/rules/penalty.md. Implement the ball-flight integrator (drag-crisis blend + Magnus) with tests.
2. Swipe-curve input fit from touch points with recorded fixtures.
3. Keeper state machine with the difficulty table; /new-minigame penalty with stadium, goal, net.
4. Alternating shoot/keep rounds + sudden death; /shot penalty.

Phase 3 Fairway Flick

Golf vs CPU — flight model + terrain



At a glance	
Phase	Phase 3
Genre	Golf vs CPU
Build	4–5 weeks
Risk	High (flight + terrain)
Rules file	.claude/rules/golf.md

In the 10-second clip (gameplay/03_fairway_flick.mp4): Swipe-swing from behind the ball; the shot climbs with a white trail, fades right in the wind, lands on the green, checks with backspin and curls along the slope grid into the cup — 'BIRDIE!'.

Physics approach

Flight = custom RK4 integrator at 240 Hz for drag and Magnus lift; hand off to a Jolt RigidBody3D (CCD on) or a custom roll model on landing.

Tuning starting points

Parameter	Value
Ball	42.67 mm, 45.93 g
Drag	$F = \frac{1}{2}\rho \cdot Cd \cdot A \cdot v^2$, Cd ≈ 0.27 low spin → ~0.32 when spin ratio S > 0.1
Lift	Cl ≈ 0.54·S^0.4, clamped to 0.25 (S = ωr/V)
Spin decay	~4% per second
Green	rolling decel ~0.6–0.9 m/s^2 + g·sin(slope) along the fall line
Cup capture	drops if centre over the cup and speed < ~1.3 m/s, else lip-out

Rules and modes

Nine compact holes built from modular tiles; stroke play vs a CPU rival; wind as a constant velocity offset in the drag term.

CPU opponent

Grid search over club/power/aim using simulated shots toward a target landing zone, plus skill noise per difficulty.

Controls and camera

Controls: Swipe-back-and-flick swing (speed = power, sideways deviation = hook/slice) or a two-tap meter; drag the aim arrow; spin dot for back/top spin.

Camera: Behind-ball aim cam → launch follow cam → landing cam; top-down 'green read' with a slope grid.

Tests Claude should write first

- Integrator: no-spin, no-wind shot is symmetric.
- Carry distance per club within ±3% of the table.
- Ball comes to rest on slopes (sleep threshold).

Pitfalls: Tunnelling into thin terrain at 70 m/s (CCD + swept raycast); jitter at rest on slopes (force sleep below a velocity threshold). **If behind:** Ship as mini-golf (rolling only, no flight model).

Path-scoped rule file: .claude/rules/golf.md

```
---
paths: ['games/golf/**']
---
- Flight: RK4 at 240 Hz. Ball 42.67 mm / 45.93 g. Cd 0.27 -> 0.32 above spin ratio 0.1; Cl = 0.54*S^0.4
    clamped 0.25;
  spin decay 4%/s; wind = constant offset in the relative velocity.
- Landing hands off to RigidBody3D with continuous_cd = true, or the custom roll model on greens.
- Force sleep below a velocity threshold on slopes; swept raycast for terrain hits at high speed.
```

Starter prompts (one per session, plan mode first)

1. Read .claude/rules/golf.md. Implement the flight integrator with Cd/Cl vs spin ratio, wind, spin decay; tests first.
2. Landing handoff to Jolt with CCD + green roll model with slope and cup capture.
3. /new-minigame golf — 3 modular holes (Kenney/Quaternius-style kit), aim arrow, swing input, cameras; then 9 holes.
4. CPU rival via simulated grid search; /physics-tune golf for carry distances.

Phase 3 Park & Pass

Original city driving & parking missions — vehicle feel + missions



At a glance	
Phase	Phase 3
Genre	Original city driving & parking missions
Build	5–7 weeks
Risk	High (vehicle feel + content)
Rules file	.claude/rules/drive.md

In the 10-second clip (gameplay/08_park_and_pass.mp4): Chase cam: the red car overtakes a slow car, stops at a red light while cross traffic passes, pulls away on green. Parking assist: reverse gear, the car swings back between the cones into bay C4 — 'PARKED! ★★★'.

Physics approach

Start with Godot's VehicleBody3D + 4× VehicleWheel3D (raycast vehicle). Plan to move to a custom raycast car (spring-damper per wheel, slip-based lateral grip) if handling feels floaty.

Tuning starting points

Parameter	Value
Mass	1200 kg, centre of mass lowered 0.3 m
Suspension	stiffness ~40–60, damping ~2–4 compression / 3–5 relaxation (Godot units)
Grip	wheel friction slip ~2.5–3.5 for arcade grip
Steering	max 0.6 rad, reduced with speed
Parking success	within 10° and 0.4 m of the bay, no cones; 3 hits = fail
Traffic	Path3D lane splines + IDM-like gap controller; pool 10–20 cars

Rules and modes

Mission types: parking, timed delivery through traffic, 'safe driver' (lane + speed limit for 60 s). Ship 10 missions, add more monthly.

CPU opponent

Traffic cars follow lane splines with a gap controller (accelerate to target speed, brake for the car ahead and for the player); intersections run light timers.

Controls and camera

Controls: Steering wheel or VirtualJoystick (Godot 4.7) on the left; gas + brake on the right; D/R gear toggle; optional tilt steering.

Camera: Chase cam with velocity look-ahead; top-down parking-assist cam.

Tests Claude should write first

- Mission detector: pose tolerances, cone hits.
- Traffic never overlaps in a 5-minute headless soak.
- Handling regression: 0–50 km/h time and braking distance within tolerance.

Pitfalls: Wheel jitter at low speed (sleep threshold, clamp tiny forces); tunnelling through thin barriers (thick colliders); frame-rate-dependent steering. **If behind:** Parking-only missions (no traffic) at first.

Path-scoped rule file: .claude/rules/drive.md

```
---
paths: ['games/drive/**']
---
- Vehicle constants live in games/drive/data/vehicle.tres: mass 1200, COM -0.3, stiffness 50, damping
    3/4,
  friction_slip 3.0, max_steer 0.6 reduced with speed.
- Missions are data (games/drive/data/missions/*.tres): type, start pose, target pose + tolerances (10
    deg, 0.4 m), cones.
- Traffic: Path3D lanes + gap controller; pooled cars, despawn behind camera. Thick colliders on
    barriers.
```

Starter prompts (one per session, plan mode first)

1. Read .claude/rules/drive.md. VehicleBody3D prototype with the tuning constants exposed in a resource; handling regression tests.
2. Parking mission schema (JSON/resource) + detector with tests.
3. Lane-spline traffic with a gap controller + light timers; 5-minute headless soak test.
4. /new-minigame drive — 5 missions, chase + parking cams, wheel/pedals UI; evaluate a custom raycast car if handling feels floaty.

Claude Code setup for this project

1. Install and connect

```
curl -fsSL https://claude.ai/install.sh | bash          # Windows: irm https://claude.ai/install.ps1 |
    iex
export GODOT=/path/to/godot-4.7                           # used by tests, hooks and MCP
claude mcp add godot -e GODOT_PATH="$GODOT" -- npx @coding-solo/godot-mcp     # run project, read debug
    output
# optional: a screenshot-capable Godot MCP (e.g. slangwald/godot-mcp) and a Blender MCP for asset
    scripting
# add --scope project to share via .mcp.json; check each server's README for current flags
```

MCP servers for Godot are community projects that change quickly; the headless CLI plus GdUnit4 (v7 runs headless by default; UI-clicking tests need xvfb-run) is the dependable core. Disable servers you aren't using to save context.

2. CLAUDE.md for the hub (in the kit)

```
# Project: Pocket Arena 8 — 3D mini-game hub (Godot 4.7.x, GDScript, Jolt)

## Golden rules
- Every game is an ORIGINAL design described by mechanics/genre. Never name, copy, or imitate a
  specific commercial game's name, logo, UI layout, art, or music. Display names live in
    games/*/manifest.tres.
- Mobile first: 60 fps target on mid-range, 30 fps floor on low-end Android; touch only.
- Game logic lives in pure GDScript classes (no Node dependencies) under games/<id>/logic/ and is unit
    tested.
- Simulation uses fixed timesteps and seeded Rng (core/Rng.gd). Never call randf() directly.
  Never scale physics by render delta.
- One game loaded at a time via SceneRouter. Games talk to the hub only through core/ autoloads.
- Save data: only write to SaveService.games[<id>]; bump that game's schema + add migrate() for any
    change.

## Commands
- Import:      godot --headless --path . --import
- Unit tests:  ./addons/gdUnit4/runtest.sh --godot_binary "$GODOT" -a res://tests -a res://games
- Run a game:  godot --path . -- --game=<id>
- Screenshot:  godot --path . -- --game=<id> --shot=user://shots/<id>.png --quit-after 300   (xvfb-run
    on Linux CI)
- Export:      godot --headless --export-release "Android" build/app.aab

## Structure
core/ autoload services · hub/ launcher, shop, settings · shared/cue_sim deterministic disc/ball solver
    .
shared/{aim_ui,power_meter,ai_common} · games/<id>/{logic,scenes,assets,tests} · ids: pool carrom golf
    penalty snake blocks chess drive

## Definition of done (every task)
1. Tests added/updated and green headless. 2. No new warnings in debug output.
3. Screenshot captured for visual changes and inspected. 4. No new per-frame allocations in
    _physics_process.
5. docs/PROGRESS.md updated (what changed, how to try it on a phone).

## Workflow
- Plan mode for any task touching >3 files or a new game; write the plan to
    docs/plans/<date>-<topic>.md.
- Small commits; one game per branch. Unfinished games stay behind enabled=false in manifest.tres.
- Design sources: docs/GDD.md (hub) and docs/games/<id>.md (per game). Visual targets:
    docs/refs/<id>/*.png
  (docs/refs/*/prototype.html are throwaway three.js mocks — never port their code).
```

Keep it under ~200 lines; topic- and path-specific guidance lives in **.claude/rules/*.md** with a `paths:` filter, so each game's rules only load when Claude touches that game. Each game chapter above prints its rule file; .claude/rules/hub.md covers core/ and hub/.

3. Skills (slash commands)

Skill	What it does	Manual only
/new-minigame	Scaffold a new mini-game folder (manifest, logic, scene, tests, rules stub, save namespace) behind a disabled feature flag.	
/physics-tune	Sweep physics parameters headlessly for one game, write a CSV and screenshots, and recommend values that meet the target metrics.	
/ai-opponent	Implement or tune a CPU opponent and report self-play win rates per difficulty.	
/perf-check	Export a release build, run the benchmark scene on a connected Android device and report frame time, draw calls and memory against budget.	yes
/shot	Capture screenshots of a game from three camera presets so Claude can inspect the 3D output.	
/release-android	Build a signed AAB and walk through the Play Console closed-testing upload.	yes

4. Subagents

Agent	Purpose	Tools
physics-reviewer	Reviews diffs in shared/cue_sim, golf, penalty and drive for frame-rate dependence, tunnelling and nondeterminism. Use after any physics change.	Read, Grep, Glob
test-writer	Writes GdUnit4 tests for pure game logic from a spec. Use when a logic class has no or weak tests.	Read, Grep, Glob, Write, Edit, Bash
ip-checker	Scans UI strings, asset names and store copy for protected names and look-alike layouts. Use before every release and after adding assets or text.	Read, Grep, Glob
asset-wrangler	Imports glTF assets, sets LODs and compression, and records licences in ASSETS.md.	Read, Grep, Glob, Write, Edit, Bash

5. Hooks (.claude/settings.json)

```
{ "hooks": {
    "PostToolUse": [{ "matcher": "Edit|Write", "hooks": [{ "type": "command", "command":
    ".claude/hooks/format_lint.sh" }] }],
    "Stop":        [{ "hooks": [{ "type": "command", "command": ".claude/hooks/test_gate.sh" }] }] } }
```

- **format_lint.sh** runs gdformat + gdlint on the edited .gd file; a lint failure exits 2 so the message reaches Claude.
- **test_gate.sh** exits 0 when `stop_hook_active` is true (no loops) or the project isn't scaffolded yet; otherwise it runs the headless GdUnit4 suite and exits 2 with the failures, so Claude keeps fixing before it stops.

6. Letting Claude see the 3D output

- Logic tests run headless.
- For visuals, run without --headless (xvfb-run on Linux CI) with a --shot argument that saves get_viewport().get_texture().get_image() to PNG and --quit-after N; Claude opens the PNGs (/shot skill).
- Add a "bot mode" autoload that plays scripted inputs for smoke tests.
- Your own thumb on a real phone twice a week: code can test green and still feel wrong.

3D asset pipeline

Source	Licence (verify before use)	Notes
Kenney.nl	CC0 — no attribution required	Car, city, minigolf and furniture kits; consistent low-poly style
Quaternius	CC0 for packs obtained before 28 Aug 2026; newer downloads under the Quaternius Asset License v1.0 (commercial use OK, no credit, but no redistribution of the raw assets)	Don't commit raw files to a public repo
Poly Pizza	Per model: CC0 or CC-BY 3.0	CC-BY needs an in-game credit line
Poly Haven	CC0 (HDRIs, textures, models)	Use downloaded files; the live API has its own attribution condition
ambientCG	CC0 — raw files may ship inside a game	Felt, wood, grass, asphalt PBR textures
Meshy (AI 3D)	Free tier CC BY 4.0 (attribution); paid tiers: you own the output	Ownership depends on your plan at generation time
Tripo (AI 3D)	Conflicting statements for the free tier	Use a paid plan for anything you ship
Audio	Kenney audio packs (CC0) + your own/commissioned music	Avoid melodies associated with competitors

Pipeline

- Blender (optionally driven by Claude via a Blender MCP server or Python scripts) → decimate to budget → bake to a shared 512–1024 px palette atlas → export glTF 2.0 (.glb) → Godot import with LODs and ETC2/ASTC compression.
- One palette and one stylised shader across all eight games keeps the collection looking like one product.
- Record every asset's source and licence in ASSETS.md (template in the kit); the ip-checker and asset-wrangler agents enforce it.
- AI 3D generators are fine for props and concepts; licence terms depend on your plan at generation time.

Phased roadmap, monetization and publishing

Phase 1, week by week (~12 weeks)

Week	Deliverable
1	Godot 4.7 project; CLAUDE.md, rules, skills, hooks, MCP; CI (headless tests + Android debug export). Create the Play Console account and start recruiting 15–20 testers.
2	Hub shell: GameRegistry, SceneRouter, SaveService v1 with migrations, Settings, Audio, InputKit, CameraKit.
3	Serpent Grid (logic, 3D render, CPU rival).
4–5	Stack Drop (reducer, kicks, bag, 3D tower, DAS/ARR, CPU rival).
6–7	Chess 3D (movegen + perft, engine, difficulty, 3D board).
8–10	shared/cue_sim + tests → Cue Club table, rules, touch UI, CPU. Start the Play closed test at week 10 with a pool-less build so the 14-day clock overlaps development.
11	Economy (coins, cosmetic skins), rewarded ads + interstitial cap, remove-ads IAP, achievements; perf pass on a low-end Android phone.
12	Closed test continues; TestFlight beta; store assets; submit v1.0.

Phase 2 (~6 weeks)

Carrom Corner (3 weeks, reuses cue_sim) + Top Corner penalty shootout (3 weeks) + daily cross-game challenges. Ship as the v1.1 update.

Phase 3 (~10 weeks)

Fairway Flick golf (4–5 weeks) + Park & Pass driving (5–7 weeks; launch with 10 missions and add more monthly). Ship as v1.2.

Monetization

- Free with ads, a one-time "Remove Ads" IAP ($2.99–$4.99), rewarded ads for coin doubling and retries, and cosmetic IAP (cue, striker, ball trail, car paint, chess set, block material).
- Cap interstitials (e.g. at most one every 3 minutes, never mid-match); no loot boxes, no energy timers.
- Premium alternative: one price (~$3.99), no ads — viable for an offline vs-CPU collection but limits reach.
- Enroll in Apple's App Store Small Business Program (15% under $1M/year); Google Play also charges 15% on the first $1M.

Store requirements

Store	What to know
Google Play	$25 one-time. Personal accounts created after 13 Nov 2023 must run a closed test with at least 12 testers opted in continuously for 14 days before applying for production, and may need to keep testing if engagement is low. Uploads must be AABs. Start the closed test around week 10 with a pool-less build.
Apple	$99/year; iOS builds need Xcode on a Mac; TestFlight for beta.

Risks, scope cuts and caveats

Game	Main risk	Cut if behind
Cue Club	Medium–High (custom physics)	Ship without spin (centre-ball only) if behind.
Stack Drop	Low physics · HIGH legal care	Drop vs-CPU; ship score-attack only.
Chess 3D	Low physics · licence care	Ship without the 3D orbit (fixed camera) and with two difficulty levels.
Serpent Grid	Low	Score-attack only (no rival).
Carrom Corner	Medium (reuses pool core)	No 'thumb' cut shots for the AI in v1.
Top Corner	Medium	Shooting only (no keeping) in the first update.
Fairway Flick	High (flight + terrain)	Ship as mini-golf (rolling only, no flight model).
Park & Pass	High (vehicle feel + content)	Parking-only missions (no traffic) at first.

Caveats

- Several tuning numbers are starting points, not measurements for your game (carrom friction, car suspension, keeper timings, soccer Magnus). Published cushion restitution values conflict; tune by feel with recorded tests.
- Godot MCP servers and Claude Code's game rule change quickly; verify current behaviour.
- Godot ad/IAP integration relies on community plugins; check maintenance status before choosing one.
- Time estimates assume ~25–30 focused hours a week with Claude Code doing most implementation; multiply by ~1.5 if you are new to Godot.
- The gameplay clips come from scripted three.js concept prototypes made to visualise each game. They are not Godot builds, and the kit tells Claude not to port their code.
- Legal points are general information, not legal advice; trademark status varies by country.

Sources

Sources used by the research behind this guide (accessed November 2026).

Game Developer — Anthropic appears to have ordered Claude to stop remaking video games — https://www.gamedeveloper.com/programming/anthropic-appears-to-have-ordered-ai-tool-claude-to-stop-remaking-video-games

Godot — maintenance release 4.7.1 — https://godotengine.org/article/maintenance-release-godot-4-7-1/

Godot — blog — https://godotengine.org/blog/

Godot — maintenance release 4.7.2 — https://godotengine.org/article/maintenance-release-godot-4-7-2/

daily.dev — post — https://daily.dev/posts/deaymlu0e

Godot — 4.7 release page — https://godotengine.org/releases/4.7/

80.lv — Godot 4.4 is here — https://80.lv/articles/godot-engine-4-4-is-here

mcpservers.org — Coding-Solo/godot-mcp — https://mcpservers.org/servers/Coding-Solo/godot-mcp

GitHub — Coding-Solo/godot-mcp — https://github.com/Coding-Solo/godot-mcp

GitHub — slangwald/godot-mcp — https://github.com/slangwald/godot-mcp

GitHub — gdUnit4-action — https://github.com/godot-gdunit-labs/gdUnit4-action

PocketGamer.biz — Unity launches official Claude Code plugin — https://www.pocketgamer.biz/unity-launches-official-claude-code-plugin-with-29-built-in-engine-skills/

Unity docs — Unity MCP get started — https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.7/manual/integration/unity-mcp-get-started.html

Kotaku — Anthropic and recreating existing games — https://kotaku.com/anthropic-nerfs-claude-again-to-stop-users-from-making-ai-slop-out-of-any-specific-existing-game-2000751665

Tom's Hardware — Claude will build an original game in the same genre — https://www.tomshardware.com/tech-industry/artificial-intelligence/anthropic-loosens-game-restrictions-claude-will-now-offer-to-build-original-game-in-same-genre-if-asked-to-copy

Loeb & Loeb — Tetris Holding v. Xio Interactive — https://www.loeb.com/en/insights/publications/2012/06/tetris-holding-llc-v-xio-interactive-inc

GitHub — official-stockfish/Stockfish — https://github.com/official-stockfish/Stockfish

Godot — 4.6 release page — https://godotengine.org/releases/4.6/

Godot docs — exporting for Android — https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_android.html

Unity blog — Unity plugin for Claude Code — https://unity.com/blog/unity-plugin-for-claude-code

Godot docs — using Jolt Physics — https://docs.godotengine.org/en/4.6/tutorials/physics/using_jolt_physics.html

Godot forum — Play Asset Delivery in Godot 4 — https://forum.godotengine.org/t/is-play-asset-delivery-supported-in-godot-4-2/55900

Iowa State — billiard trajectory analysis — https://faculty.sites.iastate.edu/jia/files/inline-files/billiard-analysis.pdf

ResearchGate — rolling ball forces figure — https://www.researchgate.net/figure/The-ball-rolling-on-the-table-It-shows-the-forces-that-are-acting-on-the-ball-while-it_fig5_228372702

Dr. Dave pool info — high-speed video study (AJP 2009) — https://drdavepoolinfo.com//physics_articles/ajp_09_hsv_article.pdf

Dr. Dave pool info — Mathavan et al. 2014 — https://drdavepoolinfo.com//physics_articles/Mathavan_Sports_2014.pdf

Dr. Dave pool info — Alciatore pool physics — https://drdavepoolinfo.com/physics_articles/Alciatore_pool_physics_article.pdf

Dr. Dave pool info — Domenech 2007 — https://drdavepoolinfo.com/physics_articles/Domenech_MechSci_07_article.pdf

ResearchGate — billiard ball dynamics under cushion impacts — https://www.researchgate.net/publication/245388279_A_theoretical_analysis_of_billiard_ball_dynamics_under_cushion_impacts

Pallis & Mehta — aerodynamics and hydrodynamics in sports — https://people.stfx.ca/smackenz/courses/HK474/Labs/Jump%20Float%20Lab/Pallis%20and%20Mehta%20%20Aerodynamics%20and%20hydrodynamics%20in%20sports.pdf

Asai et al. 2007 — fundamental aerodynamics of the soccer ball — https://people.stfx.ca/smackenz/courses/HK474/Labs/Jump%20Float%20Lab/Asai%202007%20Fundamental%20aerodynamics%20of%20the%20soccer%20ball.pdf

ResearchGate — fundamental aerodynamics of the soccer ball — https://www.researchgate.net/publication/225711147_Fundamental_aerodynamics_of_the_soccer_ball

DEV — SRS rotation, 7-bag and separating game logic — https://dev.to/sendotltd/tetris-in-vanilla-js-srs-rotation-7-bag-randomizer-and-why-you-should-separate-game-logic-from-4d25

CMU ECE design review report — http://course.ece.cmu.edu/~ece500/projects/s20-teamc1/wp-content/uploads/sites/91/2020/04/C1_DesignReview_Report.pdf

Fandom — guideline (rotation/randomizer reference) — https://tetris.fandom.com/wiki/Tetris_Guideline

Wikipedia — The Tetris Company — https://en.wikipedia.org/wiki/The_Tetris_Company

Trademark Elite — 8 BALL POOL (EU) — https://www.trademarkelite.com/europe/trademark/trademark-detail/012850764/8-BALL-POOL

Justia — 8 BALL (US application) — https://trademarks.justia.com/862/73/8-ball-86273149.html

Trademark Elite — 8 BALL POOL (UK) — https://www.trademarkelite.com/uk/trademark/trademark-detail/UK00912850764/8-BALL-POOL

Justia — 8 POOL logo (US Reg.) — https://trademarks.justia.com/791/61/8-79161387.html

Runtime Wire — Claude Code genre rule 2.1.317 — https://runtimewire.com/article/anthropic-claude-code-game-genre-rule-21317

gHacks — Claude Code rule blocks recreations of existing games — https://www.ghacks.net/2026/11/02/anthropic-adds-a-claude-code-rule-on-opus-5-5-that-blocks-recreations-of-existing-games/

Claude Code docs — features overview — https://code.claude.com/docs/en/features-overview

Claude Code docs — memory (CLAUDE.md, rules) — https://code.claude.com/docs/en/memory

Claude Code docs — .claude directory — https://code.claude.com/docs/en/claude-directory

Claude Code docs — hooks — https://code.claude.com/docs/en/hooks

Godot Awesome — Godot MCP + Claude Code setup — https://godotawesome.com/godot-mcp-claude-code-setup/

godotmcp.com — install guide — https://www.godotmcp.com/guides/install

StraySpark — Godot MCP setup with Claude Code (2026) — https://www.strayspark.studio/blog/godot-mcp-setup-claude-code-2026

Ziva — simulating input in GUT/gdUnit4 — https://ziva.sh/blogs/godot-gut-gdunit4-simulate-input

Ziva — automated playtesting in Godot — https://ziva.sh/blogs/godot-automated-playtesting

Kenney — support/licence — https://kenney.nl/support

Quaternius — licence — https://quaternius.com/license.html

Poly Pizza MCP listing — https://mcpmarket.com/server/poly-pizza

Poly Haven — licence — https://polyhaven.com/license

ambientCG — licence — https://docs.ambientcg.com/license/

Meshy — free plan — https://help.meshy.ai/en/articles/15696428-what-is-included-on-the-free-plan

Meshy — commercial use — https://help.meshy.ai/en/articles/16102098-can-i-use-meshy-assets-commercially

Tripo — pricing — https://www.tripo3d.ai/pricing

Google Play Console Help — testing requirements for new personal accounts — https://support.google.com/googleplay/android-developer/answer/14151465?hl=en